DATAWISE AI

DataWise AI: Deployment & Operations Guide

Production Readiness, Container Orchestration, Monitoring & Disaster Recovery

Document Type:	DevOps & Operational Runbook
Target Audience:	DevOps Engineers, SREs, Cloud Architects
Document Version:	v1.0.0 (Engineering Release)
Date of Issue:	October 27, 2026
Repository Status:	Verified Codebase & Production Architecture
Orchestration:	LangGraph 16-Agent Autonomous Architecture

Architectural Foundation: Deterministic ML execution engine paired with autonomous multi-agent reasoning, strict data leakage boundaries, and human-in-the-loop validation.

1. Production Deployment Topology

DataWise AI supports containerized deployment via Docker and Docker Compose, as well as cloud-native Kubernetes orchestration. The architecture decouples the stateless web/API nodes from the stateful storage and database layers.

Service Name	Runtime Container	Port	Role in Infrastructure
Service Name	Image / Runtime	Port	Operational Purpose
frontend	Nginx 1.25 + Vite Dist	80 / 443	Serves compiled SPA assets, proxies /api calls to backend
backend	Python 3.11-slim + FastAPI	8000	API routes, LangGraph agent execution, ML workers
postgres	PostgreSQL 16-alpine	5432	Primary relational database storing 37 schema entities
redis	Redis 7-alpine	6379	Distributed rate limiting, task state caching, broker
minio	MinIO S3 Compatible	9000 / 9001	Private object storage for datasets, pickles, and reports

2. Environment Variable Configuration

Production environments are configured via `.env` containing strictly sanitized references:

- `DATABASE_URL`: Connection string (`postgresql+asyncpg://user:pass@postgres:5432/datawise`).
- `STORAGE_BACKEND`: Target storage engine (`minio`, `supabase`, or `local`).
- `REDIS_URL`: Endpoint for distributed caching (`redis://redis:6379/0`).
- `LLM_PROVIDER`: Default reasoning provider (`gemini`, `groq`, `openai`, `ollama`).
- `SECRET_KEY`: High-entropy 256-bit key for local JWT signing and session validation.

3. Monitoring, Health Probes & Backup

The backend exposes `/health` and `/health/ready` endpoints verifying database connectivity, storage reachability, and Redis responsiveness. Database backups are automated using `pg_dump` snapshots, while MinIO buckets are mirrored using `mc mirror` for disaster recovery.